Jawaban UTS Pemrograman Akuntansi

Zidan Anargya Elian Mulyono

NIM 12030124130141

Program Studi S1 Akuntansi Fakultas Ekonomika dan Bisnis Universitas Diponegoro

Identitas ujian

Mata kuliah: Pemrograman / Coding. Dosen pengampu: Totok Dewayanto. Semester gasal tahun akademik 2026/2027. Tanggal pada naskah: Senin, 19 Oktober 2026. Durasi: 120 menit. Sifat: Open Book / Open AI / Open Code Sandbox.

Lingkup jawaban

Solusi terdiri atas basis data jurnal berpasangan, otomatisasi stok beserta jurnal persediaan, dan purwarupa Voucher Kas Keluar. Antarmuka dibangun dengan Streamlit; layanan transaksi menggunakan Python dan PostgreSQL, dengan pilihan API FastAPI. Database menjadi tempat penegakan aturan utama agar permintaan yang melewati antarmuka tetap diperiksa.

1. Soal 1 berbobot 25 persen: ERD, DDL, normalisasi, dan integritas jurnal.

2. Soal 2 berbobot 40 persen: trigger persediaan dan pengendalian transaksi serentak.

3. Soal 3 berbobot 35 persen: formulir web, orkestrasi AI, serta audit trail dan immutability.

Tautan artefak jawaban

URL repositori publik atau Google Drive

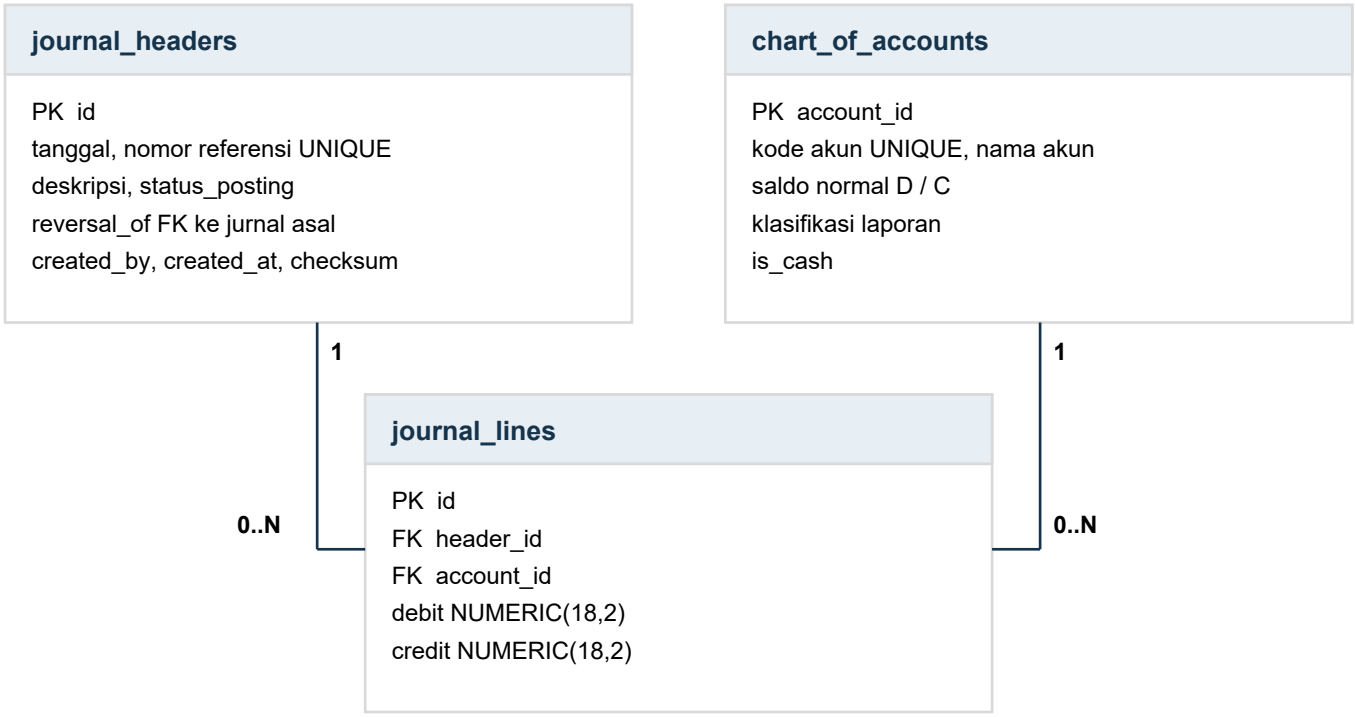
https://drive.google.com/drive/folders/1NqkymwlI97Z5JmvKt8Y5ycybTivGm1yM

Folder artefak memuat seluruh SQL, kode aplikasi, pengujian, dokumentasi prompt, dan sumber yang digunakan. Tautan repo/Drive di atas diisi sesudah unggah. Ringkasan persiapan verifikasi AI terdapat pada bagian 3.1b; bukti sesi Gemini Canvas dan NotebookLM dilampirkan dari sesi pengguna yang sebenarnya.

Soal 1 Arsitektur data

1.1 Skema relasional dan ERD

Satu header mewakili satu bukti transaksi. Detail debit/kredit disimpan sebagai beberapa baris yang merujuk header dan akun. Master akun menyimpan kode, nama, saldo normal, dan klasifikasi laporan satu kali.



Satu header dapat memiliki nol atau lebih baris selama DRAFT; saat POSTED minimal dua baris diwajibkan. Satu akun dapat dipakai pada banyak baris jurnal. Setiap baris hanya memiliki satu header dan satu akun. Relasi FK menggunakan ON DELETE RESTRICT. Kolom reversal_of menjadi relasi tambahan ke jurnal asal; UNIQUE mencegah pembalikan ganda.

1.3 Analisis normalisasi dan audit

Ketergantungan fungsional utama adalah account_id menentukan atribut akun, id header menentukan tanggal/referensi/deskripsi/status, dan id line menentukan header_id/account_id/debit/credit. Kode akun serta nomor referensi juga memiliki kekangan UNIQUE. Atribut disimpan pada entitas yang tepat sehingga tidak ada ketergantungan transitif nama/klasifikasi akun di dalam detail jurnal.

Pada tabel datar, tanggal dan deskripsi bukti berulang pada setiap baris, begitu pula nama akun jika ikut disalin. Perubahan sebagian baris dapat menghasilkan tanggal atau nama yang berbeda untuk transaksi yang sama. Penghapusan satu baris terakhir juga dapat menghilangkan informasi bukti. Pemisahan header-line mengurangi anomali pembaruan, penyisipan, dan penghapusan tersebut.

Audit lebih mudah karena satu nomor bukti mengikat semua baris, perubahan tercatat sebagai peristiwa, dan satu status posting berlaku untuk seluruh jurnal. Normalisasi saja belum menjamin jejak audit; manfaat ini harus disertai transaksi atomik, pembatasan hak akses, log, dan penguncian setelah posting.

Soal 1 DDL PostgreSQL

1.2 Tabel inti dan constraints

Berikut DDL tiga tabel inti dari sql/001_schema.sql. Versi lengkap juga menambahkan tabel persediaan, log, dan indeks. Jalankan dalam skema accounting sesuai skrip instalasi.

```
CREATE TABLE chart_of_accounts (
    account_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    account_code varchar(20) NOT NULL UNIQUE,
    account_name varchar(120) NOT NULL,
    normal_balance char(1) NOT NULL CHECK (normal_balance IN ('D','C')),
    report_class varchar(20) NOT NULL CHECK
        (report_class IN ('ASSET','LIABILITY','EQUITY','REVENUE','EXPENSE')),
    is_cash boolean NOT NULL DEFAULT false
);

CREATE TABLE journal_headers (
    id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    transaction_date date NOT NULL,
    reference_no varchar(80) NOT NULL UNIQUE CHECK (btrim(reference_no) <> ''),
    description text NOT NULL CHECK (btrim(description) <> ''),
    voucher_type varchar(20) NOT NULL DEFAULT 'GENERAL' CHECK
        (voucher_type IN ('GENERAL','CASH_OUT','INVENTORY','REVERSAL')),
    status_posting varchar(10) NOT NULL DEFAULT 'DRAFT' CHECK
        (status_posting IN ('DRAFT','POSTED')),
    reversal_of bigint UNIQUE REFERENCES journal_headers(id) ON DELETE RESTRICT,
    created_by text NOT NULL DEFAULT session_user,
    created_at timestamptz NOT NULL DEFAULT clock_timestamp(),
    posted_by text,
    posted_at timestamptz,
    hash_checksum varchar(64),
    CHECK ((voucher_type = 'REVERSAL') = (reversal_of IS NOT NULL)),
    CHECK (reversal_of IS NULL OR reversal_of <> id),
    CHECK ((status_posting = 'DRAFT' AND posted_by IS NULL
                AND posted_at IS NULL AND hash_checksum IS NULL)
        OR (status_posting = 'POSTED' AND posted_by IS NOT NULL
                AND posted_at IS NOT NULL AND hash_checksum ~ '^[0-9a-f]{64}$'))
);

CREATE TABLE journal_lines (
    id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    header_id bigint NOT NULL REFERENCES journal_headers(id) ON DELETE RESTRICT,
    account_id bigint NOT NULL REFERENCES chart_of_accounts(account_id) ON DELETE RESTRICT,
    debit numeric(18,2) NOT NULL DEFAULT 0,
    credit numeric(18,2) NOT NULL DEFAULT 0,
    CHECK (debit >= 0 AND debit < 'Infinity'::numeric),
    CHECK (credit >= 0 AND credit < 'Infinity'::numeric),
    CHECK ((debit > 0 AND credit = 0) OR (credit > 0 AND debit = 0))
);
```

NOT NULL menutup celah nilai NULL pada CHECK. Aturan satu sisi positif memperkuat syarat soal: debit/kredit tidak negatif, tidak keduanya nol, serta tidak keduanya positif. Nilai disimpan sebagai NUMERIC; input aplikasi dibatasi dua angka desimal. [4]

Soal 1 Integritas pada saat posting

Keseimbangan lintas baris

CHECK pada satu baris tidak cukup untuk menjamin jumlah seluruh debit sama dengan kredit. PostgreSQL tidak mendukung CHECK lintas baris sebagai jaminan konsistensi agregat. Karena itu, total diperiksa oleh trigger sebelum status beralih menjadi POSTED. Draft boleh belum lengkap; hanya POSTED dimasukkan ke tampilan buku besar. [4]

```
SELECT count(*), COALESCE(sum(debit),0), COALESCE(sum(credit),0)
INTO n,d,c
FROM accounting.journal_lines WHERE header_id = NEW.id;
IF n < 2 OR d <= 0 OR d <> c THEN
  RAISE EXCEPTION 'Jurnal tidak seimbang';
END IF;
```

Fungsi post_journal mengunci header dengan FOR UPDATE. Semua INSERT/UPDATE/DELETE detail mengambil lock pada header yang sama sebelum perubahan. Dengan urutan tersebut, pengeditan detail tidak bisa menyelinap selama posting. Jurnal baru selalu DRAFT, sehingga INSERT langsung berstatus POSTED juga ditolak.

Fungsi submit_cash_voucher menyisipkan header dan semua baris, lalu memanggil post_journal dalam satu transaksi. Error pada salah satu tahap membatalkan header, detail, dan audit log operasi tersebut. Referensi UNIQUE mengurangi risiko pencatatan ganda saat permintaan diulang. Backend tidak menyediakan perubahan untuk jurnal POSTED.

Pengembangan konteks persediaan

Soal 2.1 secara eksplisit meminta perubahan stok dan penolakan stok minus. Implementasi juga membuat jurnal karena nama fungsi dan konteks soal menekankan sinkronisasi persediaan dan HPP. Asumsi tambahan ditetapkan sebagai berikut.

Peristiwa	Stok	Debit	Kredit
PURCHASE	Bertambah qty	Persediaan	Utang usaha
SALE	Berkurang qty	HPP	Persediaan
ADJUSTMENT positif	Bertambah qty	Persediaan	Selisih persediaan
ADJUSTMENT negatif	Berkurang abs(qty)	Selisih persediaan	Persediaan

Nilai jurnal = abs(perubahan stok) x unit_cost. Biaya per unit dianggap tetap, bukan algoritma FIFO/rata-rata tertimbang. PURCHASE diasumsikan kredit. SALE hanya membentuk jurnal HPP; jurnal pendapatan memerlukan harga jual yang tidak tersedia pada skema soal. Untuk pembelian tunai dapat digunakan PURCHASE kemudian voucher pelunasan utang. ADJUSTMENT bertanda harus disertai alasan.

Soal 2 Trigger persediaan

2.1 Pembaruan stok dan jurnal secara atomik

Trigger BEFORE INSERT mengunci barang sebelum memeriksa saldo. Jika jurnal gagal diposting atau stok tidak mencukupi, INSERT dan seluruh efeknya dibatalkan. Berikut implementasi dari sql/002_business_logic.sql.

```
CREATE FUNCTION trg_update_stock_and_journal() RETURNS trigger LANGUAGE plpgsql SECURITY DEFINER
SET search_path = pg_catalog, accounting AS $$
DECLARE current_qty int; cost numeric; delta bigint; amount numeric;
        hid bigint; debit_code text; credit_code text;
BEGIN
 -- BEFORE INSERT: lock diambil sebelum FK INSERT memperoleh KEY SHARE.
 SELECT stock_qty,unit_cost INTO current_qty,cost
 FROM accounting.inventory_items WHERE item_id=NEW.item_id FOR UPDATE;
 IF NOT FOUND THEN RAISE EXCEPTION 'Barang tidak ditemukan'; END IF;
 IF NEW.qty IS NULL OR NEW.trx_type IS NULL
    OR NEW.trx_type NOT IN ('PURCHASE','SALE','ADJUSTMENT')
    OR (NEW.trx_type IN ('PURCHASE','SALE') AND NEW.qty <= 0)
    OR (NEW.trx_type='ADJUSTMENT' AND NEW.qty=0) THEN
   RAISE EXCEPTION 'Jenis atau kuantitas transaksi tidak valid';
 END IF;
 delta := CASE WHEN NEW.trx_type='SALE' THEN -NEW.qty::bigint ELSE NEW.qty::bigint END;
 IF current_qty::bigint + delta < 0 THEN
   RAISE EXCEPTION 'Stok tidak cukup: tersedia %, perubahan %',current_qty,delta;
 END IF;
 UPDATE accounting.inventory_items SET stock_qty=stock_qty+delta WHERE item_id=NEW.item_id;
 NEW.unit_cost_snapshot := cost;
 NEW.created_by := session_user; NEW.trx_date := clock_timestamp();
 amount := abs(delta)*cost;
 IF NEW.trx_type='PURCHASE' THEN debit_code:='1103'; credit_code:='2101';
 ELSIF NEW.trx_type='SALE' THEN debit_code:='5101'; credit_code:='1103';
 ELSIF delta>0 THEN debit_code:='1103'; credit_code:='5102';
 ELSE debit_code:='5102'; credit_code:='1103'; END IF;
 INSERT INTO accounting.journal_headers(transaction_date,reference_no,description,voucher_type)
 VALUES((NEW.trx_date AT TIME ZONE 'Asia/Jakarta')::date,'INV-'||NEW.reference_no,
        NEW.trx_type||' '||NEW.item_id||': '||NEW.reason,'INVENTORY') RETURNING id INTO hid;
 INSERT INTO accounting.journal_lines(header_id,account_id,debit,credit)
 SELECT hid,account_id,CASE WHEN account_code=debit_code THEN amount ELSE 0 END,
        CASE WHEN account_code=credit_code THEN amount ELSE 0 END
 FROM accounting.chart_of_accounts WHERE account_code IN (debit_code,credit_code);
 PERFORM accounting.post_journal(hid);
 NEW.journal_id:=hid;
 RETURN NEW;
END $$;
CREATE TRIGGER update_stock_and_journal BEFORE INSERT ON inventory_transactions
FOR EACH ROW EXECUTE FUNCTION trg_update_stock_and_journal();
```

Soal 2 Transaksi serentak

2.2 Race condition dan pencegahan overselling

Tanpa lock, kasir A dan B dapat sama-sama membaca stok 1, lalu keduanya menganggap penjualan sah. Solusi menggunakan READ COMMITTED dengan SELECT FOR UPDATE pada baris barang sebelum pengecekan dan pengurangan. Lock bertahan sampai COMMIT/ROLLBACK. Kasir kedua menunggu, lalu memeriksa versi stok setelah perubahan kasir pertama. [5]

Urutan	Kasir A	Kasir B
1	Mengunci barang; membaca stok 1	Meminta lock barang yang sama
2	Mencatat SALE 1; stok menjadi 0	Menunggu transaksi A
3	COMMIT	Mendapat lock; membaca stok 0
4	Satu penjualan tersimpan	RAISE EXCEPTION; ROLLBACK

```
-- Sesi A
BEGIN;
SELECT accounting.record_inventory(
  'RACE001','SALE',1,'RACE-A','Penjualan kasir A');
-- Jangan commit dahulu; jalankan perintah Sesi B.
COMMIT;

-- Sesi B, terminal terpisah
BEGIN;
SELECT accounting.record_inventory(
  'RACE001','SALE',1,'RACE-B','Penjualan kasir B');
-- Menunggu A; kemudian error stok tidak cukup.
ROLLBACK;
```

Persiapan barang dan stok awal satu unit tersedia pada sql/005_concurrency_demo.sql. Hasil uji otomatis memverifikasi wait_event_type = Lock untuk koneksi kasir B sebelum A commit. Setelah selesai, stok 0 dan tepat satu SALE tersimpan. Ini adalah uji dua koneksi PostgreSQL nyata.

Alternatifnya gunakan SERIALIZABLE dan ulangi seluruh transaksi ketika terjadi SQLSTATE 40001. Menambah tingkat isolasi tanpa mekanisme retry belum cukup untuk memberikan pengalaman aplikasi yang baik. Error deadlock 40P01 juga dapat memerlukan retry; error stok tidak cukup adalah penolakan bisnis, bukan alasan mengulang tanpa perubahan data. [6]

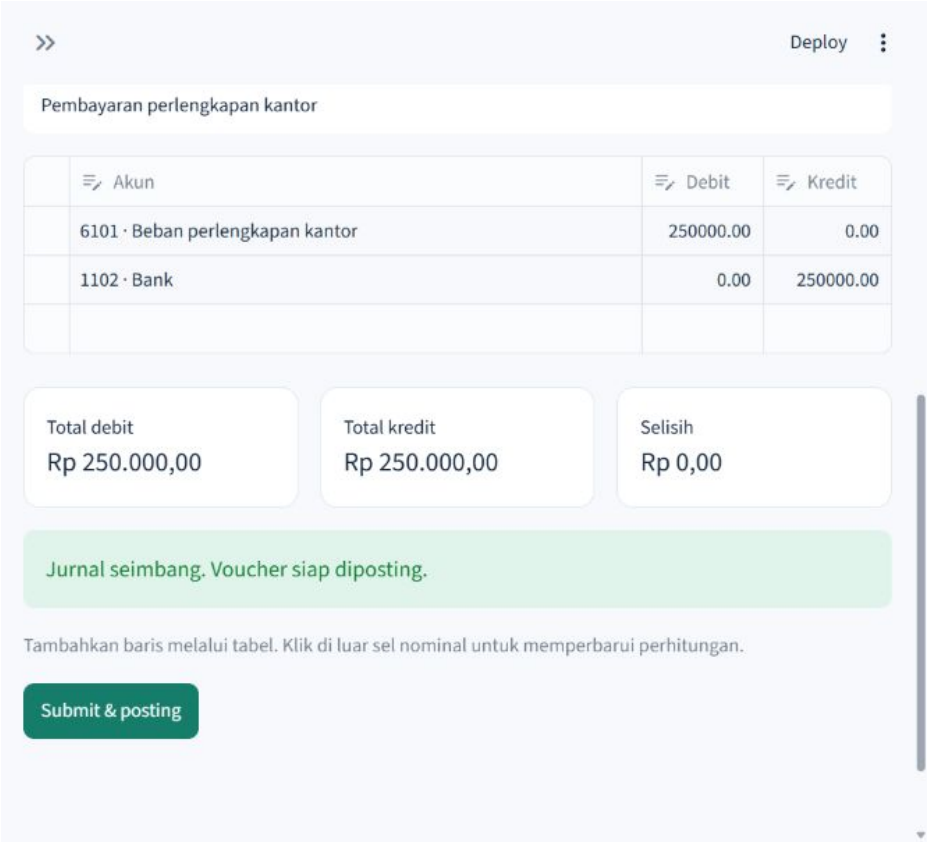
Untuk transaksi multi-item, ambil lock barang dalam urutan item_id yang konsisten agar risiko deadlock berkurang. Kasus ini adalah race condition/overselling pada baris yang sama; istilah phantom stock pada soal tidak harus identik dengan fenomena phantom read dalam teori isolasi.

Soal 3 Purwarupa web

3.1a Formulir dan indikator keseimbangan

Purwarupa menyediakan tanggal, nomor bukti, deskripsi, serta tabel akun/debit/kredit yang dapat ditambah dan dihapus. Total diperbarui saat perubahan sel dikomit. Nominal dihitung dengan Decimal di backend. Submit aktif jika minimal dua baris valid, jumlah debit sama dengan kredit dan positif, serta metadata lengkap.

Khusus CASH_OUT, harus terdapat kredit kas/bank tanpa debit kas. Perubahan persediaan dilakukan melalui modul stok. Backend dan database mengulang pemeriksaan; menyiasati tampilan tidak membuat transaksi tidak valid dapat diterima.



Gambar 2. Formulir yang dijalankan lokal menunjukkan debit dan kredit Rp250.000, selisih nol, serta tombol posting aktif. Tampilan ini adalah bukti aplikasi, bukan tangkapan layar Gemini Canvas.

Kode implementasi berada dalam app.py, backend/validation.py, dan backend/service.py. Sumber koneksi memakai DATABASE_URL di server sehingga password tidak dikirim ke frontend. Streamlit menggunakan layanan Python yang sama dengan API FastAPI. Skema `accounting` telah dipasang pada proyek Supabase khusus UTS; uji SQL cloud satu sesi lulus 12 pemeriksaan dan membatalkan seluruh data ujinya. [7][8]

Soal 3 Orkestrasi dan verifikasi AI

Dokumentasi prompt Gemini Canvas

Prompt lengkap berada dalam docs/prompt_gemini_canvas.txt. Instruksi intinya meminta formulir Streamlit dengan baris dinamis, total debit-kredit otomatis, tombol bersyarat, validasi di server, serta pembatasan transaksi POSTED. Kode proyek diberikan sebagai konteks agar rancangan dapat dibandingkan dengan implementasi yang diuji.

Log bantuan Codex yang benar-benar terjadi disimpan pada docs/log_bantuan_ai.md. Prompt dijalankan di Gemini Canvas pada 13 Oktober 2026 dan menghasilkan artifact `app.py`. Review pertama dikoreksi karena mengubah nilai kosong menjadi nol; revisi akhir mempertahankan validasi strict, menolak akun kosong/tidak dikenal, dan mereset editor setelah posting. Bukti serta artifact hasil Canvas disimpan dalam docs/bukti_ai/.

3.1b Verifikasi NotebookLM

Sesi NotebookLM dijalankan pada 13 Oktober 2026 dengan sumber IAI, materi IAS 7/IFRIC, dan kode proyek. Respons bersitasi membedakan aturan PSAK/IAS 7 dari kontrol aplikasi. Hasilnya menegaskan bahwa keseimbangan jurnal, kunci POSTED, reversal, SHA-256, dan audit log adalah kontrol aplikasi; semuanya tidak sendirinya membuktikan kepatuhan PSAK 207. Ringkasan final dan bukti sitasi disimpan dalam docs/bukti_ai/.

PSAK 207 merupakan penomoran baru PSAK 2 sejak 1 Januari 2024 dan mengatur laporan arus kas. [1] Kas mencakup saldo kas dan rekening giro. Setara kas merupakan investasi jangka pendek yang sangat likuid, dapat dikonversi ke jumlah kas tertentu, serta memiliki risiko perubahan nilai yang tidak signifikan. [2] Bahan IAS 7 menekankan jatuh tempo singkat sejak perolehan; kriteria sekitar tiga bulan tidak dipakai sendirian untuk mengklasifikasikan instrumen. [3]

Pemeriksaan	Implementasi dan penilaian awal
Akun kas yang diizinkan	Master demo menandai Kas dan Bank. Investasi tidak otomatis ditandai setara kas.
Kas keluar	Debit mengikuti substansi transaksi; kredit mengurangi kas/bank.
Transaksi nonkas	PURCHASE kredit tidak dimasukkan sebagai voucher kas keluar.
Saldo berpasangan	Database menolak ketidakseimbangan; belum membuktikan substansi transaksi benar.
Audit trail	POSTED, reversal, dan checksum adalah kontrol sistem; bukan klaim teknik yang diwajibkan PSAK 207.

Verifikasi final harus menilai bukti transaksi, klasifikasi arus kas, dan standar pengakuan/pengukuran yang relevan. Kode ini tidak menghasilkan laporan arus kas lengkap dan tidak mengotomatisasi penilaian semua instrumen keuangan. Tidak ada klaim kepatuhan menyeluruh hanya karena total jurnal seimbang.

Soal 3 Backend dan immutability

3.2 Spesifikasi fungsi transaksi

Backend Python/FastAPI memanggil fungsi PostgreSQL melalui query berparameter. Transaksi pada psycopg melakukan commit setelah seluruh operasi berhasil dan rollback ketika exception muncul. Jalur UI dan HTTP menggunakan logika layanan yang sama; status POSTED selalu dikunci oleh trigger database.

Antarmuka	Fungsi dan hasil
POST /vouchers	submit_cash_voucher; menerima tanggal, referensi, deskripsi, baris; menghasilkan ID jurnal POSTED.
POST /inventory	record_inventory; trigger mengubah stok dan membentuk jurnal pada transaksi yang sama.
POST /journals/{id}/reverse	reverse_journal; menukar debit/kredit pada jurnal baru dan menautkan reversal_of.
GET /journals/{id}/integrity	verify_journal; membandingkan checksum tersimpan dan hasil perhitungan ulang.
PUT/DELETE /journals/{id}	Ditolak dengan 405; koreksi menggunakan endpoint reversal.

```
# Inti layanan Python, backend/service.py
with connection() as conn:
    return conn.execute(
        "SELECT accounting.submit_cash_voucher(%s,%s,%s,%s) AS id",
        (trx_date, ref.strip(), description.strip(), Jsonb(clean))
    ).fetchone()["id"]
```

```
-- Kontrol pada header, berlaku meskipun UI dilewati
IF OLD.status_posting = 'POSTED' THEN
  RAISE EXCEPTION
    'Jurnal POSTED tidak boleh diubah/dihapus; gunakan reversal';
END IF;
```

Pembalik menyimpan tanggal dan alasan koreksi, tidak mengubah status atau isi jurnal asal. UNIQUE pada reversal_of mencegah duplikasi. Pembalik tidak boleh bertanggal sebelum jurnal asal. Jurnal stok dikoreksi melalui ADJUSTMENT agar kuantitas dan buku besar tetap sinkron, bukan dengan reversal finansial terpisah. Jurnal pembalik tidak dibalik ulang pada purwarupa ini.

Grup uts_accounting_app hanya memiliki SELECT dan EXECUTE atas empat fungsi API yang diperlukan. Tidak ada izin langsung UPDATE/DELETE/INSERT/TRUNCATE tabel. Fungsi SECURITY DEFINER mengunci search_path ke pg_catalog dan accounting; PUBLIC EXECUTE dicabut. Kredensial runtime berbeda dari administrator.

Soal 3 Metadata dan checksum

Jejak audit dan verifikasi integritas

created_by ditetapkan database dari session_user, bukan kolom teks bebas dari browser. created_at menggunakan waktu server dengan timestamptz. posted_by dan posted_at merekam tahap posting. Purwarupa satu pengguna memakai login database khusus; bila diperluas menjadi multiuser, identitas pengguna akhir harus berasal dari autentikasi backend yang diverifikasi.

Tabel audit_log mencatat aktor, waktu, jenis operasi, nama tabel, kunci record, JSON sebelum/sesudah, dan transaction_id. Trigger mencatat perubahan header, detail, barang, dan transaksi persediaan. Log bersifat append-only untuk jalur aplikasi. Peristiwa yang ikut rollback tidak tersimpan; log kegagalan akses/transaksi memerlukan logging server terpisah.

hash_checksum menyimpan SHA-256 payload header dan semua detail terurut id. Kolom hash sendiri dikeluarkan dari payload. Timestamp dinormalisasi ke UTC dengan format tetap agar verifikasi tidak bergantung pada zona waktu sesi.

```
NEW.hash_checksum := encode(
  sha256(convert_to(
    accounting.journal_payload(NEW)::text, 'UTF8'
  )), 'hex'
);
```

Saat verifikasi, database membangun ulang payload dengan representasi yang sama dan membandingkan hash. Ketidaksamaan menunjukkan bahwa isi jurnal tidak lagi cocok dengan checksum. Hasil uji juga memeriksa kestabilan hash pada UTC, Asia/Jakarta, dan America/New_York.

Hash bukan enkripsi, tanda tangan digital, atau bukti bahwa transaksi ekonominya sah. Master akun tidak termasuk dalam hash header-detail. Pengguna runtime tidak dapat mengubah master; perubahan oleh administrator harus dikelola terpisah. Administrator/superuser yang mengendalikan data dan kode dapat menonaktifkan kontrol atau mengganti hash. Untuk ancaman tersebut diperlukan log/cadangan eksternal atau tanda tangan dengan kunci yang dikelola terpisah.

Contoh pembalikan

Voucher awal: debit Beban perlengkapan Rp250.000 dan kredit Bank Rp250.000. Jurnal pembalik: debit Bank Rp250.000 dan kredit Beban perlengkapan Rp250.000. Jumlah neto per akun untuk pasangan jurnal tersebut menjadi nol. Kedua bukti tetap tersedia untuk ditelusuri; koreksi yang benar dapat dicatat sebagai voucher baru.

Pengujian dan bukti pelaksanaan

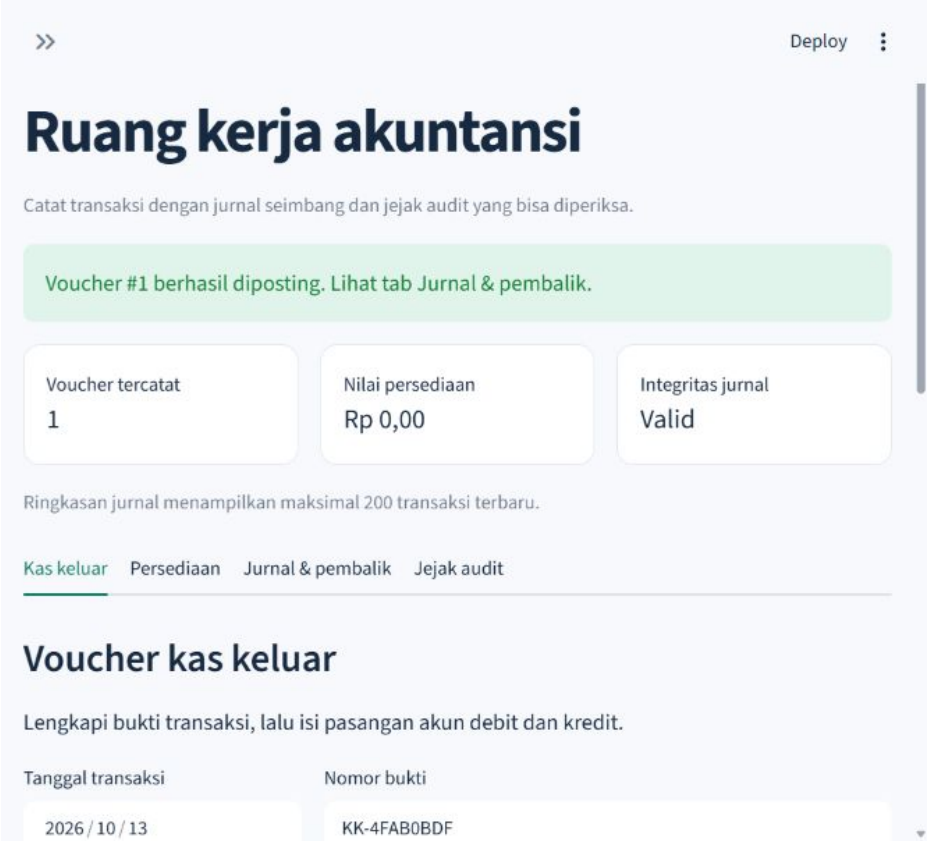
Lingkungan dan hasil

Pengujian lokal menggunakan Python 3.12 dan PostgreSQL 16.15 pada cluster terpisah. Backend dijalankan dengan role aplikasi non-owner. Hasil berikut diambil dari docs/hasil_uji.json. Selain itu, `sql/006_cloud_smoke_test.sql` dijalankan melalui SQL Editor proyek Supabase PostgreSQL 17.11: 12 pemeriksaan lulus dan data uji di-rollback. Uji cloud tersebut tidak diklaim sebagai tes concurrency dua koneksi atau koneksi runtime aplikasi.

35 pengujian dijalankan; 0 gagal; 0 error; 0 dilewati.

Waktu eksekusi UTC: 2026-10-13T07:29:10.079010+00:00

Skenario meliputi nominal negatif/nol/ganda/NULL, FK dan ON DELETE RESTRICT, penolakan jurnal tidak seimbang, penolakan edit/hapus POSTED, pembalik, duplikasi nomor bukti, audit append-only, stok minus, ADJUSTMENT bertanda, privilege, checksum lintas zona waktu, dan dua kasir pada stok terakhir. Uji UI memeriksa Submit, metadata wajib, perubahan saldo, dan penyimpanan voucher. API diperiksa untuk autentikasi, payload tidak valid, dan larangan perubahan.



Gambar 3. Posting melalui browser berhasil menghasilkan voucher nomor 1 pada database demo. Nama/NIM terdapat pada sidebar aplikasi. Data pada gambar adalah contoh pengujian, bukan transaksi keuangan nyata.

Panduan pengumpulan dan referensi

Menjalankan dan melengkapi artefak

README memuat pemasangan Python, instalasi SQL, pembuatan role aplikasi, koneksi Supabase, serta langkah demo. Urutan SQL adalah 001_schema, 002_business_logic, 003_seed, kemudian 004_permissions. Gunakan database proyek baru; installer menolak menimpa skema accounting yang sudah ada.

Sesi Gemini Canvas dan NotebookLM telah dijalankan, bukti asli/ringkasan disimpan, dan skema cloud dipasang pada proyek Supabase khusus UTS. Kode/dokumentasi diunggah ke folder Drive yang dapat dilihat melalui link, tanpa password atau secret. URL pengumpulan tercantum pada halaman pertama PDF dan submission.json. Komting mengumpulkan rekap tautan paling lambat 15 menit setelah ujian selesai sesuai petunjuk naskah.

Referensi

[1] IAI. PSAK 207 Laporan Arus Kas dan perubahan penomoran dari PSAK 2. https://web.iaiglobal.or.id/PSAK-Umum/8

[2] IAI. Modul CAFB Akuntansi Keuangan 2025. Halaman tampilan 163, nomor tercetak 151. Definisi kas dan setara kas. https://web.iaiglobal.or.id/assets/materi/Sertifikasi/CA/modul/ak/files/basic-html/page163.html

[3] IFRS Interpretations Committee. IFRIC Update Januari 2013. IAS 7 Identification of cash equivalents. https://media.ifrs.org/2013/IFRIC/January/IFRIC-Update-January-2013.html

[4] PostgreSQL. Constraints. https://www.postgresql.org/docs/current/ddl-constraints.html

[5] PostgreSQL. Explicit Locking. https://www.postgresql.org/docs/current/explicit-locking.html

[6] PostgreSQL. Transaction Isolation. https://www.postgresql.org/docs/current/transaction-iso.html

[7] Streamlit. st.data_editor. https://docs.streamlit.io/develop/api-reference/data/st.data_editor

[8] Supabase. Connect to your database. https://supabase.com/docs/guides/database/connecting-to-postgres

Referensi diperiksa pada 13 Oktober 2026. Teks PSAK lengkap mengikuti akses resmi IAI atau materi dosen. Bahan pendamping IFRS digunakan untuk konsep; penilaian final tetap memerlukan sumber standar yang sesuai untuk entitas dan periode pelaporan.